

Разработка интеллектуального модуля интерактивного анализа данных в реляционных базах данных

Выпускная квалификационная работа

Студент: Гимазетдинов Дмитрий Русланович

Московский авиационный институт
(Национальный исследовательский университет)

2026

Проблема:

- Бизнес-цели требуют быстрого отклика от аналитиков
- Аналитики вынуждены ждать разработчиков для получения отчётов
- Прослойка между аналитиком и данными становится рудиментом

Пример:

«Какие самолёты чаще всего летали из Москвы в Санкт-Петербург в прошлом месяце с загрузкой более 80%?»
— требует нескольких JOIN и знания схемы

Факторы актуальности:

- 1 Экспоненциальный рост объёмов данных
- 2 Высокий порог входа в аналитику (SQL)
- 3 Активное развитие LLM-моделей
- 4 Ограничения существующих решений

Решение: NLIDB-интерфейс на основе RAG + LLM + агентный подход + режим самокоррекции

Цель и задачи исследования

Цель:

Разработать программное решение, которое устранил барьер между пользователями без технического образования и сложными реляционными базами данных.

Задачи:

- 1 Провести оценку современных методов трансляции естественного языка в SQL
- 2 Спроектировать RAG-архитектуру для работы со схемой БД (метаинформация, связи таблиц)
- 3 Разработать механизм самокоррекции ИИ-агента (Self-Heal)
- 4 Провести оценку качества генерации до и после самокоррекции
- 5 Сравнить LLM-провайдеры: GLM-4.7, Qwen3-Coder-Next, GigaChat-2
- 6 Реализовать микросервисную архитектуру системы

Сравнение с существующими решениями

Функционал	Vanna.AI	LangChain	Metabase	Наш продукт
Генерация SQL (Text-to-SQL)	+	+	+	+
RAG для схемы БД	+	Ограничено	--	+
Локальный запуск (Privacy)	+	+	--	+
Механизм самокоррекции (Self-Heal)	--	--	--	+
Авто-визуализация данных	±	--	+	В разработке

Ключевые преимущества нашего решения

- **Приватность:** можно все вычисления вести локально, данные не покидают контур
- **Self-Heal:** автоматическая итеративная самокоррекция SQL
- **RAG для сложных схем:** оптимизировано для многотабличных БД

Микросервисы:

- **Frontend** — веб-интерфейс пользователя
- **RAG Service** — оркестратор пайплайна
- **Agent Module** — LangGraph агент (Chain-of-Thought)
- **Embedding Model Service** — локальная векторизация (multilingual-e5-base)
- **Retrieval Pipeline** — семантический поиск в ChromaDB
- **Ingest Pipeline** — индексация документов (Redis Queue)
- **DB Connector Service** — безопасный доступ к PostgreSQL

Хранилища:

- **MinIO** — объектное хранилище файлов (S3-совместимо)
- **ChromaDB** — векторная БД
- **PostgreSQL** — реляционная БД
- **Redis** — очередь задач

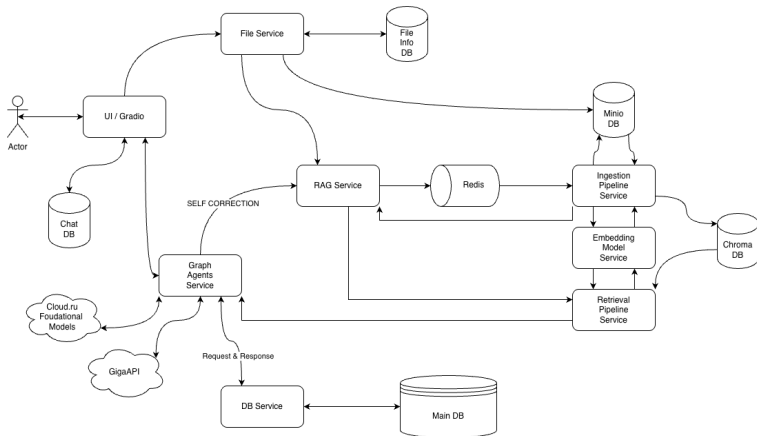


Рис.: HLD диаграмма архитектуры системы

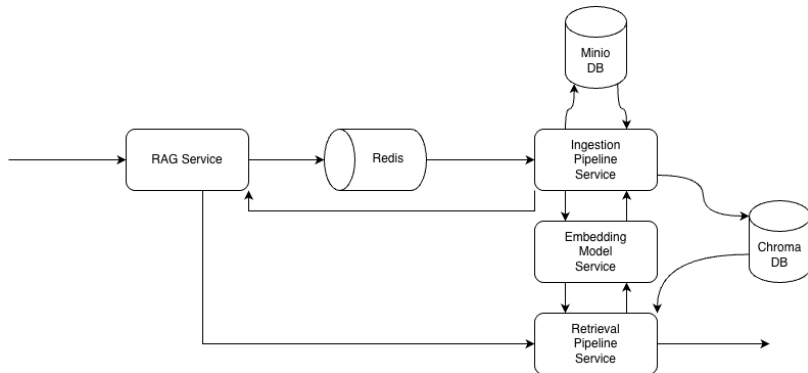


Рис.: RAG-пайплайн

Агентный цикл (LangGraph):

- 1 Переписывание запроса для RAG-поиска
- 2 Семантический поиск контекста схемы БД
- 3 Генерация SQL на основе контекста
- 4 Выполнение SQL в DB Connector Service
- 5 **Критика результата** — LLM анализирует ошибку и выносит вердикт
- 6 Если нужно: запрос схемы проблемных таблиц
- 7 Улучшение SQL с учётом замечаний
- 8 Повтор до 5 итераций

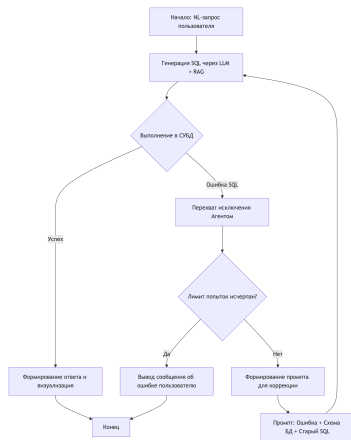


Рис.: Алгоритм самокоррекции

Метрики оценки качества генерации SQL

Execution Accuracy (EA):

$$EA = \frac{|\{q : \text{exec}(q) = \text{exec}(q^*)\}|}{|Q|}$$

Доля запросов, результат выполнения которых совпал с эталоном. Совпадение результирующих множеств.

Exact Matching (EM):

$$EM = \frac{|\{q : q = q^*\}|}{|Q|}$$

Точное совпадение SQL текста.

Exact Set Matching (ESM):

$$ESM = \frac{|\{q : R(q) \equiv R(q^*)\}|}{|Q|}$$

Сравнивает структурные компоненты SQL-запроса (например, списки столбцов, условия фильтрации, ...)

Intellect (IEA):

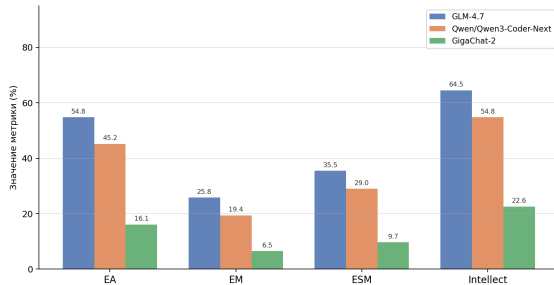
$$IEA = \frac{|\{q : \text{exec}(q) \approx \text{exec}(q^*)\}|}{|Q|}$$

Учитывает семантически верные, но синтаксически отличающиеся запросы.

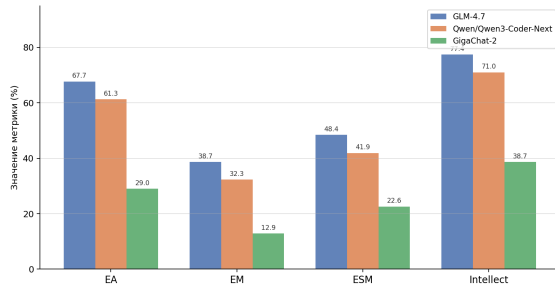
Тестовый набор: **31 SQL-задача** разного уровня сложности на БД «Авиаперевозки»

Результаты сравнения LLM-моделей

Метрики моделей до самокоррекции



Метрики моделей после самокоррекции



Результаты сравнения LLM-моделей

Без самокоррекции

Модель	EA	EM	ESM	IEA
GLM-4.7	54,8	25,8	35,5	64,5
Qwen3-Coder	45,2	19,4	29,0	54,8
GigaChat-2	16,1	6,5	9,7	22,6

С самокоррекцией

Модель	EA	EM	ESM	IEA
GLM-4.7	67,7	38,7	48,4	77,4
Qwen3-Coder	61,3	32,3	41,9	71,0
GigaChat-2	29,0	12,9	22,6	38,7

Прирост EA от самокоррекции

	GLM-4.7	Qwen3-Coder	GigaChat-2
Δ EA	+13,0 п.п.	+16,1 п.п.	+12,9 п.п.

Вывод: GLM-4.7 лидирует по всем метрикам в обоих режимах. Qwen демонстрирует наибольший относительный прирост от самокоррекции. GigaChat-2 существенно уступает.

Технические характеристики:

- Векторизация запроса: **80–150 мс**
- Семантический поиск (10К чанков): **50–100 мс**
- Полный агентный цикл (1–3 итерации): **15–45 сек**
- Индексация документа (10 стр.): **20–40 сек**

Развёртывание:

- Одна команда: `docker compose up -d`
- Веб-интерфейс: `localhost:3000`
- API: `localhost:8003`
- Поддерживаемые форматы: PDF, DOCX, TXT

Область применения:

- Розничная торговля и логистика
- Финансовый сектор
- Производство и телеком
- Авиаперевозки
- Банковские хранилища

Апробация:

Тестирование на демонстрационной БД «Авиаперевозки» (Postgres Pro):

- 8 взаимосвязанных таблиц
- JSONB-поля с мультиязычными атрибутами
- 31 SQL-задача разного уровня сложности

Научная новизна:

- Адаптивный метод семантического поиска контекста схемы БД с многоуровневым индексированием (структура + комментарии + примеры данных)
- Оригинальный алгоритм многошаговой самокоррекции SQL («критик» + «исполнитель»)

Конфиденциальность данных:

- Все вычисления могут исполняться локально, внутри корпоративного контура
- Модели эмбедингов запускаются на собственном сервере
- Данные не передаются во внешние облачные сервисы

Масштабируемость:

- Горизонтальное масштабирование каждого сервиса независимо
- ChromaDB: от десятков тысяч до миллионов векторов
- Замена LLM или модели эмбедингов без переработки системы

Пример работы системы

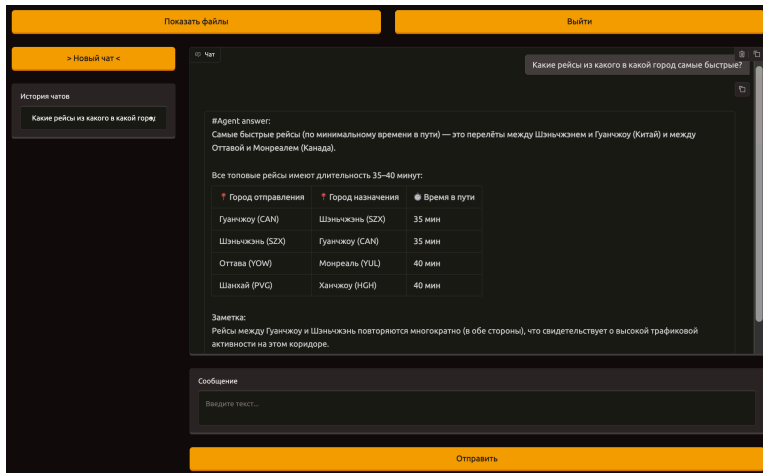


Рис.: UI



Рис.: GitHub



Рис.: GitVerse